

# Tabelas *Hash*

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

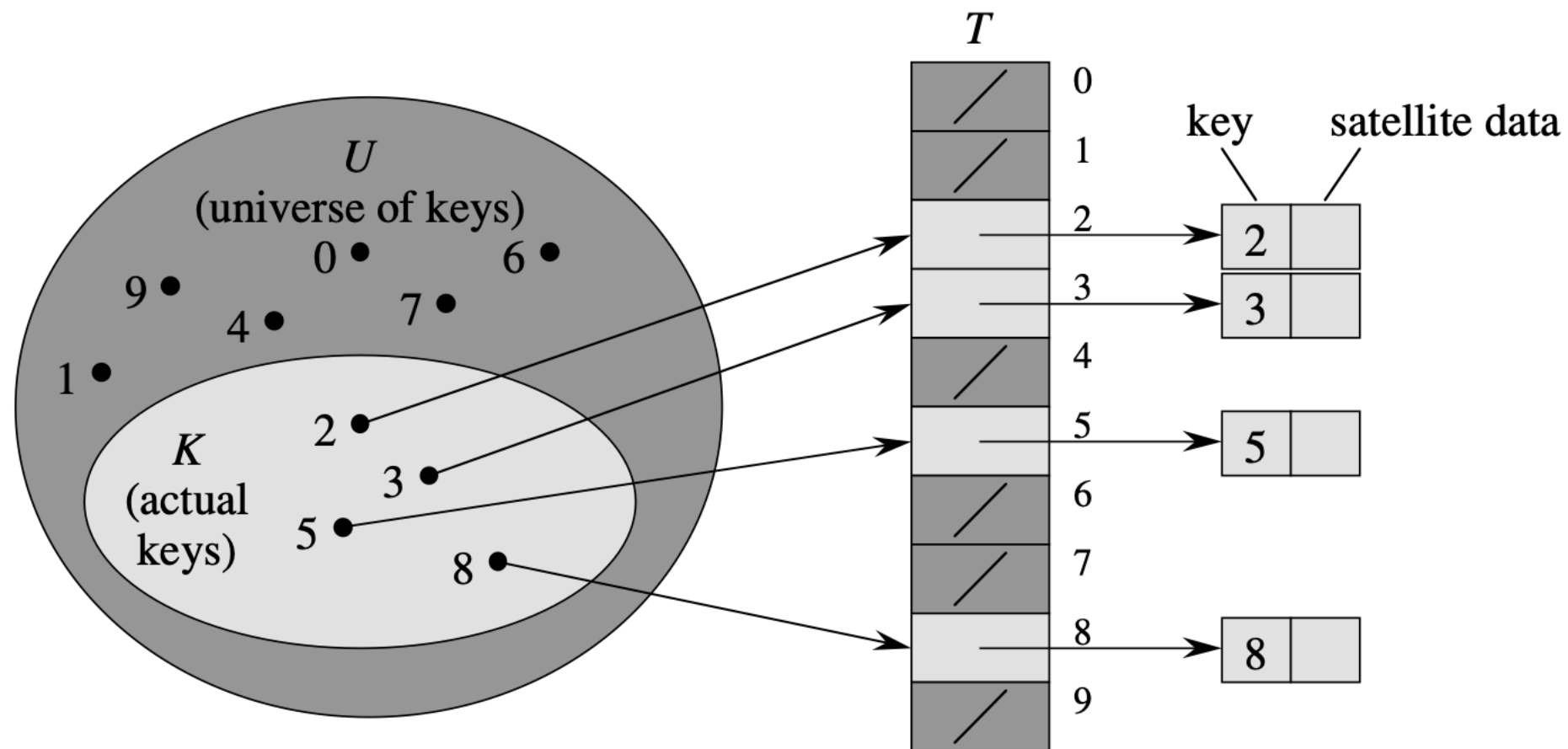
# Introdução

- Muitas aplicações requerem um conjunto dinâmico que suporte apenas as operações de dicionário INSERIR, BUSCAR e REMOVER
  - Por exemplo, um compilador para um linguagem de computação mantém uma tabela de símbolos
- **Tabela hash** é uma estrutura de dados efetiva para implementar dicionários
  - Embora buscar um elemento na tabela pode levar no pior caso tempo  $\Theta(n)$
  - Na prática, sob hipóteses razoáveis, o tempo esperado para a busca por um elemento na tabela é  $O(1)$

# Tabela de Endereçamento Direto

- **Endereçamento direto** é uma técnica que funciona bem quando o universo  $U$  de chaves é razoavelmente pequena
- Suponha aplicação necessite de um conjunto dinâmico no qual cada elemento tem uma **chave** obtida do universo  $U = \{0, 1, \dots, m - 1\}$ 
  - $m$  não é muito grande
- Assuma nenhum par de elementos tenham a mesma **chave**
- Para representar o conjunto dinâmico, utilizaremos um arranjo, ou **tabela de endereçamento direto**, denotado por  $T[0 \dots m - 1]$ , na qual cada **posição**, ou **slot**, corresponde a uma chave no universo  $U$

# Figura: Endereçamento Direto



Implementando um conjunto dinâmico com uma tabela de acesso direto  $T$ . Cada chave no universo  $U = \{0, 1, \dots, 9\}$  corresponde a um índice na tabela. O conjunto  $K = \{2, 3, 5, 8\}$  de chaves reais determinam os slots na tabela que contém um ponteiro para os elementos. Os outros slots, fortemente sombreado, contém NULL. Fonte: CORMEN (2012).

# Endereçamento Direto: Operações

**DIRECT-ADDRESS-SEARCH**( $T, k$ )

**return**  $T[k]$

**DIRECT-ADDRESS-INSERT**( $T, x$ )

$T[key[x]] \leftarrow x$

**DIRECT-ADDRESS-DELETE**( $T, x$ )

$T[key[x]] \leftarrow \text{NIL}$

- Cada uma destas operações é rápida: apenas requer tempo  $O(1)$

# Tabelas Hash

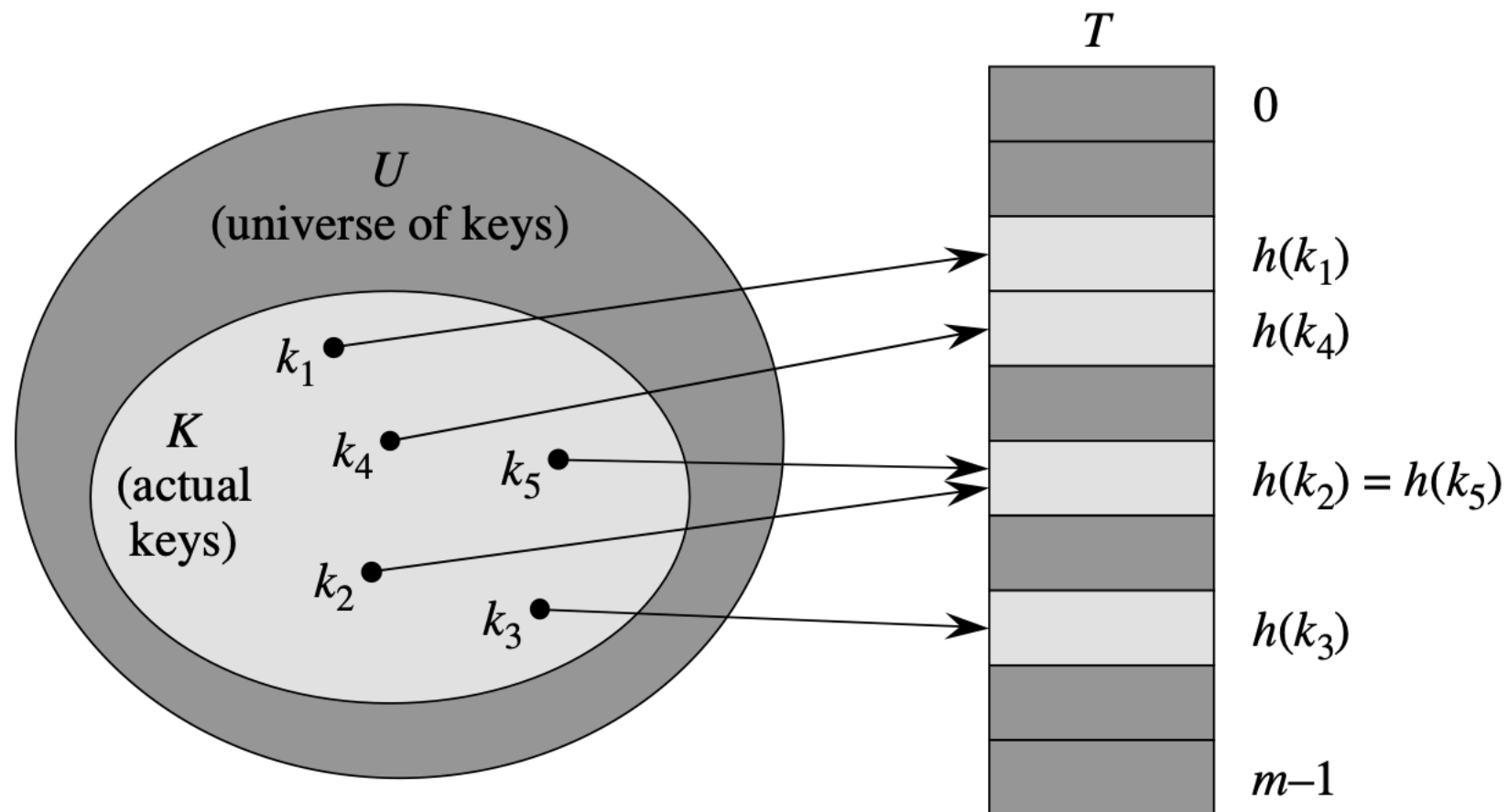
- Limitações do **endereçamento direto**:
  - Se  $U$  é muito grande, armazenar tabela  $T$  de tamanho  $|U|$  é inviável (ou mesmo impossível)
  - Além disso, o conjunto  $K$  de chaves realmente armazenadas pode ser pequeno em relação a  $U$ , de tal forma que grande parte do espaço alocado para  $T$  seria desperdiçado
- Quando  $K \ll U$ , uma **tabela hash** requer menos espaço que uma **tabela de endereçamento direto**
  - De fato, os requisitos de armazenamento  $\rightarrow \Theta(|K|)$  enquanto nós mantemos o benefício de buscar um elemento na tabela hash em tempo  $O(1)$  no caso médio

# Tabela de Endereçamento Direto vs. Hash

- Para a **tabela de endereçamento direto** isso se mantém para o **pior caso**
- Com **endereçamento direto**:
  - Elemento de chave  $k$  é armazenado no slot  $k$
- Com **hashing**:
  - Elemento é armazenado na posição (slot)  $h(k)$
  - Uma **função hash**  $h$  computa a posição (slot) a partir da chave  $k$   
 $h : U \rightarrow [0, 1, \dots, m - 1]$



# Figura: Função Hash



Usando função hash  $h$  para mapear chaves para slots na tabela hash. Chaves  $k_2$  e  $k_5$  são mapeadas para o mesmo slot, então elas colidem. Fonte: CORMEN (2012).



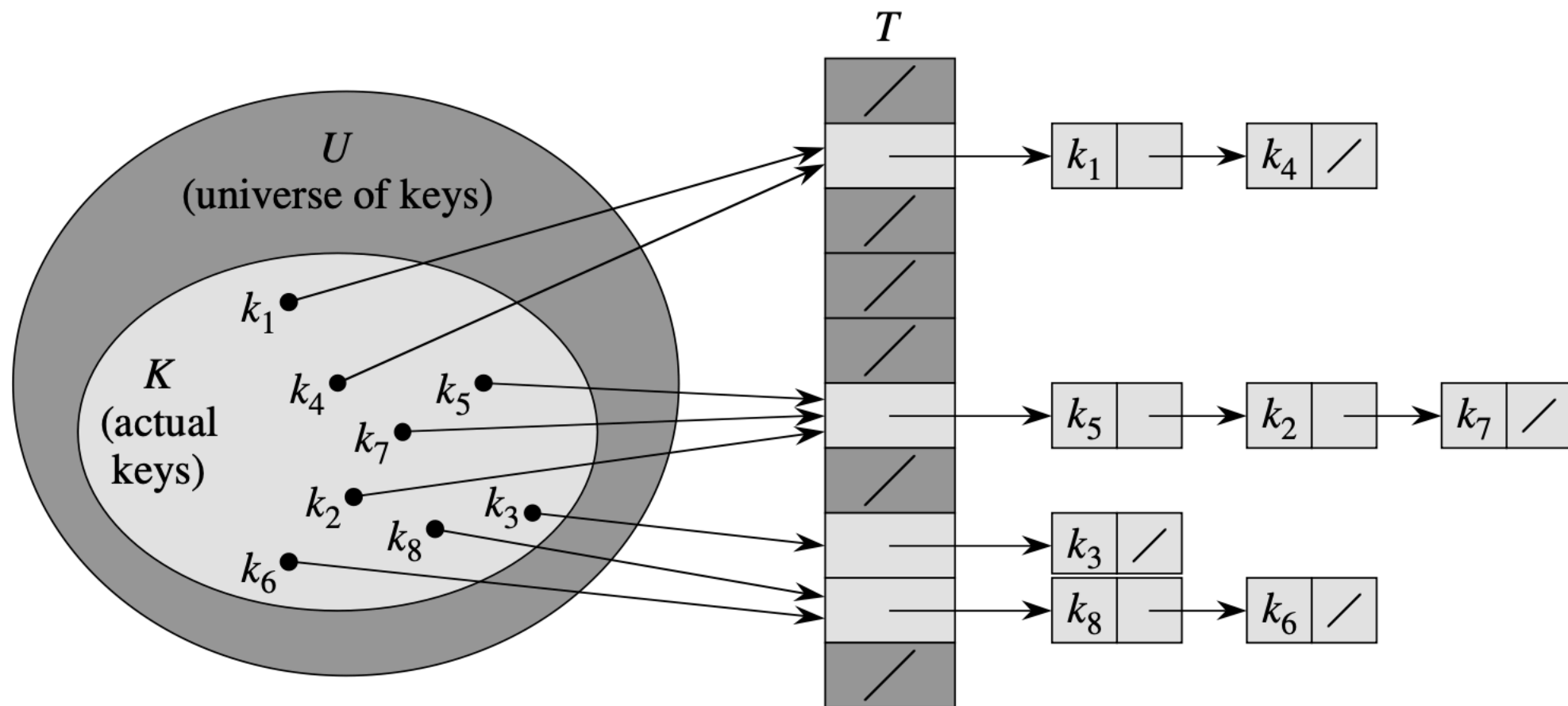
# Colisão

- Duas chaves podem ser **mapeadas** para a mesma posição (*slot*)
  - Chamamos esta situação de **colisão**
- Existem técnicas para resolver conflito criado pela colisão
- É claro que uma solução ideal é evitar colisão
  - Podemos tentar atingir isso escolhendo uma **função hash**  $h$  adequada
  - Uma ideia é fazer  $h$  parecer “aleatória”, assim evitando colisões ou pelo menos minimizando o seu número

# Resolução de Colisões por Encadeamento

- No **encadeamento**, colocamos todos os elementos cujo valor **hash** é o mesmo (ou seja, mapeado para o mesmo slot) em uma **lista encadeada**
  - Posição (*slot*)  $j$  contém um ponteiro para o cabeçalho da lista de todos os elementos armazenados que possuem o valor hash  $j$
  - Se não houver tais elementos, o slot  $j$  contém *NULL*

# Figura: Encadeamento



Tratamento de colisões por encadeamento. Cada slot na tabela  $T[j]$  contém uma lista encadeada de todas as chaves cujo valor hash é  $j$ . Por exemplo,  $h(k_1) = h(k_4)$  e  $h(k_5) = h(k_2) = h(k_7)$ . Fonte: CORMEN (2012).

# Encadeamento: Operações

CHAINED-HASH-INSERT( $T, x$ )

insert  $x$  at the head of list  $T[h(key[x])]$

CHAINED-HASH-SEARCH( $T, k$ )

search for an element with key  $k$  in list  $T[h(k)]$

CHAINED-HASH-DELETE( $T, x$ )

delete  $x$  from the list  $T[h(key[x])]$

# Encadeamento: Desempenho

- Quão bem **hashing** com encadeamento desempenha?  
Quanto tempo leva para buscar um elemento dado uma chave??!!
- Dado uma tabela de dispersão  $T$  com  $m$  posições (*slots*) que armazenam  $n$  elementos, nós definimos o **fator de carga**  $\alpha$  para  $T$  como  $n/m$   
 $\Rightarrow$  isto é, o número médio de elementos armazenados no encadeamento
  - Nossa análise será em termos de  $\alpha$ , que pode ser menor, igual, ou maior que 1

# Encadeamento: Desempenho (Cont.)

- O tempo total requerido para uma **busca com sucesso** é  $\Theta(1 + \alpha)$  (para quem tiver interesse, a prova está no livro do Cormen)
- O que isso significa?
  - Se o número de slots é pelo menos proporcional ao número de elementos na tabela, nós temos  $n = O(m)$  e, consequentemente,  $\alpha = n/m = O(m)/m = O(1)$
  - Assim, buscar é  $O(1)$  **na média**
  - Uma vez que a inserção leva  $O(1)$  no **pior caso**
  - E remoção leva  $O(1)$  no **pior caso** quando lista duplamente encadeada
  - Todas as operações de dicionário levam tempo  $O(1)$  **na média**

# O Que Faz Uma Função Hash Boa?

- Uma boa **função hash** satisfaz (aproximadamente) a suposição de **hashing uniforme simples**:
  - Cada chave tem mesma probabilidade de ser associada a qualquer uma das  $m$  posições (*slots*)
  - Independentemente de onde qualquer outra chave tenha sido associada
- Infelizmente, não é possível verificar esta condição
- Na prática, técnicas heurísticas são frequentemente utilizadas para criar **funções hash** com bom desempenho



# Interpretando Chaves Como Números Naturais

- Maioria das **funções hash** assumem que o **universo de chaves** é o conjunto  $\mathbf{N} = \{0, 1, 2, \dots\}$  de números naturais
- Assim, se **chaves** não são números naturais, é preciso encontrar uma maneira de interpretá-las como números naturais



# Interpretando Chaves Como Números Naturais (Cont.)

- Por exemplo, uma cadeia de caracteres pode ser interpretada como um inteiro expresso em uma notação decimal adequada
- $pt$  pode ser interpretado como um par de inteiros decimais  $(112, 116)$ , uma vez que  $p = 112$  e  $t = 116$  em ASCII
  - Expresso como um inteiro de base 128,  $pt$  se torna  $(112 \cdot 128) + 116 = 14452$

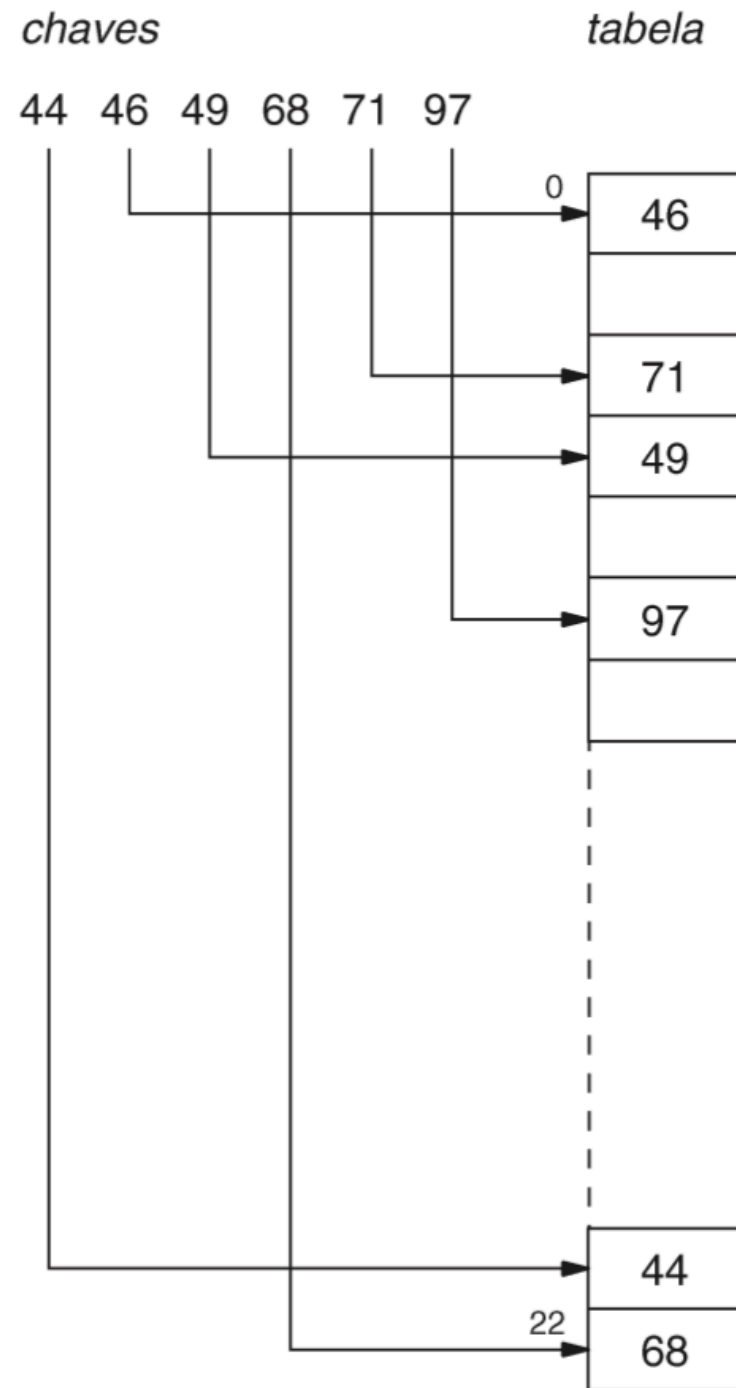
# Método da Divisão

- No **método da divisão**, mapeamos a chave  $k$  em uma das  $m$  posições (*slots*) pegando o resto da divisão de  $k$  dividido por  $m$
- Isto é, **função hash** é:

$$h(k) = k \bmod m .$$

- Por exemplo, se a tabela hash tem tamanho  $m = 12$  e a chave é  $k = 100$ , então  $h(k) = 4$ 
  - Uma vez que requer apenas uma única operação de divisão, a **transformação da chave** pela divisão é rápida

# Figura: Método da Divisão



Seja uma tabela hash com  $m = 23$ . Fonte:  
SZWARCFITER (2010).

# Método da Divisão: Considerações

- Alguns valores de  $m$  são melhores que outros
- Por exemplo,  $m$  não deve ser potência de 2, uma vez que se  $m = 2^p$ , então  $h(k)$  é simplesmente os  $p$  bits de ordem mais baixa de  $k$
- Suponha  $m = 16 = 2^4$ . Então:  $h(k) = k \bmod 16$

| k (decimal) | k (binário) | últimos 4 bits | h(k) |
|-------------|-------------|----------------|------|
| 26          | 11010       | 1010           | 10   |
| 42          | 101010      | 1010           | 10   |
| 58          | 111010      | 1010           | 10   |

- Uma boa escolha consiste em escolher  $m$  de modo que seja um número primo não próximo a uma potência de 2

# Método da Multiplicação

- Opera em dois passos:
  1. Multiplicamos a chave  $k$  por uma constante  $A$  no intervalo  $0 < A < 1$  e extraímos a parte fracionária de  $kA$
  2. Em seguida, multiplicamos este valor por  $m$  e o tomamos o piso do resultado

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

(Knuth sugere  $A \approx (\sqrt{5} - 1)/2 = 0.6180339887$ )

# Método da Multiplicação: Exemplo 1

- **Exemplo:**  $m = 1000$ ,  $k = 123$ ,  $A \approx 0.6180339887\dots$

$$\begin{aligned} h(k) &= \lfloor 1000(123 \cdot 0.6180339887 \bmod 1) \rfloor \\ &= \lfloor 1000 \cdot 0.018169\dots \rfloor = 18. \end{aligned}$$

# Método da Multiplicação: Considerações

- **Vantagens:**
  - Valor de  $m$  não é crítico
  - Tipicamente escolhido como potência de 2, ou seja,  $m = 2^p$ , o que facilita a implementação
- **Desvantagens:**
  - Mais lento que o método da divisão



# Endereçamento Aberto (*Open Addressing*)

- No **endereçamento aberto**, todos os elementos são armazenados na própria **tabela hash**
  - Cada entrada na tabela contém um elemento ou *NULL*
- Não há listas e nenhum elemento armazenado fora da tabela, como há em encadeamento
- Tabela pode “encher” de tal forma que nenhuma outra inserção possa ser feita
  - $0 \leq \alpha \leq 1$ : o fator de carga  $\alpha$  nunca pode exceder 1



# Endereçamento Aberto (Cont.)

- Ao **procurar** por um elemento, examinamos sistematicamente as posições (*slots*) da tabela até que:
  - o elemento desejado seja encontrado
  - ou fique claro que o elemento não está na tabela
- Em vez de seguir os ponteiros, calculamos a sequência de posições (*slots*) a serem examinadas

# Endereçamento Aberto: Inserção

- Realizamos uma **sequência de sondagem** (*probing*) na tabela hash até encontrarmos uma posição (*slot*) vazia para colocar a chave
- Em vez de ser fixo na ordem  $0, 1, \dots, m - 1$  (que requer  $\Theta(n)$  tempo de pesquisa), a sequência de posições sondadas depende da chave que está sendo inserida

# Endereçamento Aberto: Inserção (Cont.)

- Estendemos a **função hash** para incluir o **número da sondagem** (*probe number*), começando de 0, como uma segunda entrada. Assim, a função hash se torna:

$$h : U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\} .$$

- Com o **endereço aberto**, exigimos que, para cada chave  $k$ , a **sequência de sondagem** (*probe sequence*)

$$\langle h(k,0), h(k,1), \dots, h(k, m - 1) \rangle$$

seja uma permutação de  $\langle 0, 1, \dots, m - 1 \rangle$ .

# Endereçamento Aberto: Pseudocódigo Inserção

**HASH-INSERT**( $T, k$ )

```
1   $i \leftarrow 0$   
2  repeat  $j \leftarrow h(k, i)$   
3      if  $T[j] = \text{NIL}$   
4          then  $T[j] \leftarrow k$   
5              return  $j$   
6          else  $i \leftarrow i + 1$   
7  until  $i = m$   
8  error “hash table overflow”
```

# Endereçamento Aberto: Pseudocódigo Busca

HASH-SEARCH( $T, k$ )

```
1   $i \leftarrow 0$ 
2  repeat   $j \leftarrow h(k, i)$ 
3           if  $T[j] = k$ 
4             then return  $j$ 
5            $i \leftarrow i + 1$ 
6  until  $T[j] = \text{NIL}$  or  $i = n$ 
7  return NIL
```

# Endereçamento Aberto: Remoção

- Quando **removemos uma chave** da posição (*slot*)  $i$ , não podemos simplesmente marcar essa posição (*slot*) como vazio armazenando *NULL*
  - Por quê??!!
  - Pode tornar impossível recuperar qualquer chave  $k$  durante cuja inserção sondamos a posição (*slot*)  $i$  e a encontramos ocupada
- Uma **solução** é marcar o *slot* armazenando nele o valor especial *REMOVIDO* em vez de *NULL*
  - É necessário modificar algoritmo HASH-SEARCH

# Endereçamento Aberto: Remoção (Cont.)

- Quando usamos o valor especial *REMOVIDO*, no entanto, os **tempos de pesquisa** não dependem mais do fator de carga  $\alpha$
- Por isso **encadeamento** é mais comumente selecionado como uma técnica de resolução de colisão quando as chaves devem ser excluídas!!



# Hashing Uniforme

- Anteriormente, fizemos a suposição de **hash uniforme**
  - Cada chave é igualmente provável de ter qualquer uma das  $m!$  permutações de  $\langle 0, 1, \dots, m - 1 \rangle$  como sua **sequência de sondagem**
- **Hashing uniforme verdadeira** é difícil de implementar, e na prática aproximações adequadas (como o hashing duplo, que veremos a seguir) são usadas



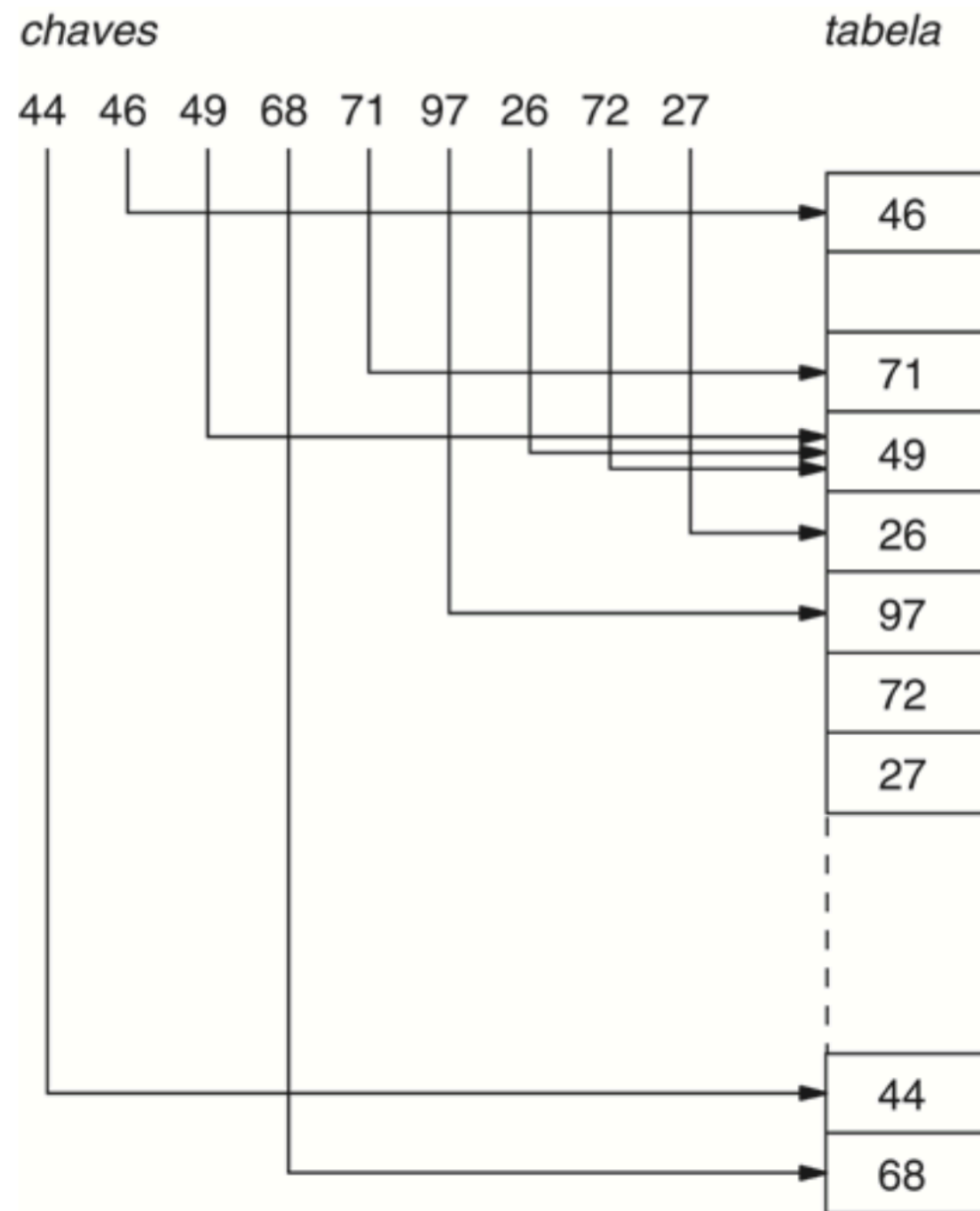
# Sondagem Linear

- Dada uma **função hash comum**  $h' : U \rightarrow \{0, 1, \dots, m - 1\}$ , que referimos como uma **função hash auxiliar**, o **método de sondagem linear** usa a função hash:

$$h(k, i) = (h'(k) + i) \bmod m$$

- para  $i = 0, 1, \dots, m - 1$ .

# Figura: Sondagem Linear



$h(k) = (h'(k) + i) \bmod 23$ .  $h'(k) = k \bmod 23$  .  
Fonte: SZWARCFITER (2010).

# Sondagem Linear (Cont.)

- Como a **sondagem inicial** determina toda a sequência da sondagem
  - Gera  $m$  sequências de sondagens distintas!!
- **Sondagem linear** é fácil de implementar
- Mas sofre de um problema conhecido como **agrupamento primário**
  - Longas sequências de posições (*slots*) ocupados se acumulam, aumentando o tempo médio de busca

# Sondagem Quadrática

- A **sondagem quadrática** usa uma função hash da forma:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m,$$

- Onde  $h'$  é uma **função hash auxiliar**,  $c_1$  e  $c_2 \neq 0$  são constantes auxiliares, e  $i = 0, 1, \dots, m - 1$

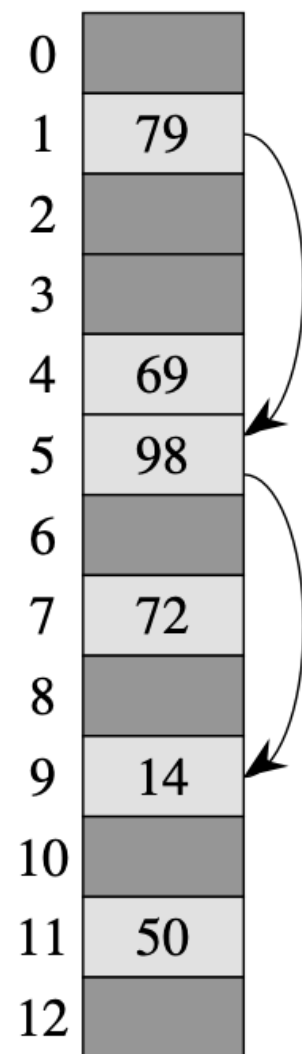
# Sondagem Quadrática (Cont.)

- Funciona melhor do que **sondagem linear**, mas para fazer pleno uso da tabela hash, os valores de  $c_1$ ,  $c_2$  e  $m$  são restritos
- Se duas chaves tiverem mesma posição inicial da sondagem, então suas sequências de sonda são as mesmas, já que  $h(k_1, 0) = h(k_2, 0)$  implica  $h(k_1, i) = h(k_2, i)$ 
  - Essa propriedade leva a uma forma mais suave de agrupamento, chamada de **agrupamento secundário**
- Gera  $m$  sequências de sondagens distintas!!

# Hash Duplo

- Um dos melhores métodos disponíveis para endereçamento aberto (gera  $\Theta(m^2)$  **sequências de sondagens**)
  - Permutações produzidas têm muitas das características de permutações escolhidas aleatoriamente
- O **hash duplo** usa uma função hash da forma:
$$h(k, i) = (h_1(k) + ih_2(k)) \bmod m,$$
onde  $h_1$  e  $h_2$  são **funções hash auxiliares**
- Ao contrário do caso de **sondagem linear** ou **quadrática**, a sequência da sonda aqui depende de duas maneiras da chave  $k$

# Inserção por Hash Duplo



Inserção por hash duplo. Aqui nós temos uma tabela hash de tamanho 13 com  $h_1(k) = k \bmod 13$  e  $h_2(k) = 1 + (k \bmod 11)$ . Uma vez que  $14 \equiv 1(\bmod 13)$  e  $14 \equiv 3(\bmod 11)$ , a chave 14 é inserida no slot vazio 9, depois de slots 1 e 5 serem examinados e encontrados ocupados. Fonte: CORMEN (2012).



# Referências

- CORMEN, T. H. [et al]. Algoritmos: teoria e prática. 3a ed. Rio de Janeiro: Elsevier, 2012
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estrutura de dados e seus algoritmos, 3 ed. Rio de Janeiro, RJ: LTC, 2010. 302p